Ascent
FORMATION

FORMATION PROFESSIONNELLE · COURS APPRENANT

Cours apprenant

ReST API — bonnes pratiques et sécurité

MODULE 2
Conception sécurisée des API
Authentification, autorisation, validation des entrées, OAuth2, JWT

CRÉATEUR DU COURS
Boris Rose
Formateur & concepteur du contenu

Table des matières

Avant-propos **2**

1 Pourquoi sécuriser une API ? **2**

2 La surface d'attaque d'une API REST **2**

3 OWASP API Security Top 10 (édition 2023) **2**

4 Authentification et autorisation : deux questions distinctes **3**

5 Les méthodes d'authentification **3**
5.1 HTTP Basic — pour mémoire . . . 3
5.2 API Keys . . . 3
5.3 OAuth 2.0 et OpenID Connect . . . 4
5.4 Le flow Authorization Code + PKCE (étape par étape) . . . 4

6 JWT : tokens auto-portés **5**
6.1 Structure d'un JWT . . . 5
6.2 Claims standards . . . 5
6.3 Algorithmes . . . 5
6.4 Pièges JWT — les classiques . . . 5

7 Validation des entrées **6**
7.1 Tout ce qui vient du client est suspect . . . 6
7.2 Valider avec un schéma déclaratif . . . 6

8 Les injections **6**
8.1 Injection SQL . . . 7
8.2 Autres injections à connaître . . . 7

9 Les outils du module **7**
9.1 jwt.io . . . 7
9.2 Auth0 . . . 7
9.3 Keycloak . . . 8
9.4 Ajv, Joi, Zod, Pydantic . . . 8

10 Synthèse **8**

11 Lexique du module **8**

12 Pour aller plus loin **9**

Avant-propos

Ce cours complète les diapositives du Module 2 en explicitant chaque terme technique, en illustrant chaque concept par un exemple concret, et en présentant les outils que vous utiliserez en travaux pratiques.

1 Pourquoi sécuriser une API ?

Les API exposent directement la logique métier et les données. Contrairement à une page web qui peut filtrer ce qui est affiché, une API renvoie toujours ce qu'elle renvoie : une mauvaise gestion des permissions devient immédiatement une fuite massive.

À retenir

Les API représentent aujourd'hui plus de 80 % du trafic web mondial. 9 risques sur 10 du Top OWASP API portent sur l'authentification, l'autorisation ou la validation. Aucun client n'accepte une fuite de données : la sécurité n'est pas négociable.

2 La surface d'attaque d'une API REST

Définition

Surface d'attaque : ensemble des points par lesquels un attaquant peut potentiellement interagir avec un système. Plus la surface est large, plus elle est difficile à défendre.

Sept couches à inspecter pour une API REST :

- **Transport** : TLS, certificats, négociation, attaques de l'homme du milieu (*MITM*).
- **Authentification** : vol de credentials, brute force, replay, OAuth mal configuré.
- **Autorisation** : accès à des objets non autorisés, élévation de privilège.
- **Entrées** : injections SQL/NoSQL/XML/JSON, désérialisation non sûre, mass assignment.
- **Sorties** : fuites de données, messages d'erreur trop verbeux.
- **Logique métier** : détournement de workflow, conditions de course (*race conditions*).
- **Infrastructure** : CORS large, secrets en clair, dépendances obsolètes.

3 OWASP API Security Top 10 (édition 2023)

Définition

OWASP (*Open Web Application Security Project*) est une fondation à but non lucratif qui publie depuis 2001 des guides de référence pour la sécurité applicative. Le *OWASP API Security Top 10* liste les 10 risques les plus critiques spécifiquement pour les API.

Voici les 10 catégories de risques (édition 2023). Ne retenez ici que les noms et les concepts : nous reverrons les détails au Module 3.

1. **API1 – BOLA** : Broken Object Level Authorization. Accès à un objet appartenant à un autre utilisateur.

2. **API2 – Broken Authentication** : Sessions, JWT, tokens mal protégés.
3. **API3 – BOPLA** : Broken Object Property Level Auth. Lecture/écriture de champs interdits.
4. **API4 – Unrestricted Resource Consumption** : Pas de rate limiting, requêtes coûteuses non plafonnées.
5. **API5 – BFLA** : Broken Function Level Authorization. Endpoint admin accessible par un user normal.
6. **API6 – Unrestricted Access to Sensitive Flows** : workflow critique sans garde-fou.
7. **API7 – SSRF** : Server-Side Request Forgery. L'API émet des requêtes pour le compte de l'attaquant.
8. **API8 – Security Misconfiguration** : CORS large, debug actif, headers manquants.
9. **API9 – Improper Inventory Management** : endpoints fantômes, vieilles versions exposées.
10. **API10 – Unsafe Consumption of APIs** : confiance aveugle envers une API tierce.

4 Authentification et autorisation : deux questions distinctes

Définition

Authentification : « qui es-tu ? ». Vérifier l'identité du client.

Autorisation : « qu'as-tu le droit de faire ? ». Décider si l'identité authentifiée a accès à une ressource ou peut effectuer une action.

L'erreur classique consiste à confondre les deux. Un utilisateur peut être *authentifié* mais ne pas être *autorisé* pour une action donnée. Les codes HTTP les distinguent :

À retenir

401 Unauthorized = pas authentifié (le serveur ne sait pas qui vous êtes).
403 Forbidden = authentifié mais sans permission (le serveur sait qui vous êtes et refuse).

5 Les méthodes d'authentification

5.1 HTTP Basic — pour mémoire

Authentification par couple identifiant/mot de passe encodé en Base64 dans l'en-tête `Authorization`. Très simple, mais à *proscrire en production publique* (le mot de passe est rejoué à chaque requête). Utilisable pour des outils internes derrière HTTPS.

5.2 API Keys

Définition

API Key : chaîne de caractères longue et aléatoire (typiquement 32+ octets) qui identifie un client. Transmise dans un en-tête (`Authorization` ou `X-API-Key`).

Quand l'utiliser ? Pour identifier un client *programmatique* (script, intégration, partenaire) sans utilisateur final humain. Permet de faire du rate limiting, des quotas, de la facturation par clé.

Bonnes pratiques.

- Toujours en en-tête, jamais dans la query string (logguée par les serveurs et le navigateur).
- Stocker côté serveur uniquement le *hash* de la clé (bcrypt ou argon2), pas la clé en clair.
- Rotation régulière, révocation immédiate possible.
- Préfixer pour faciliter la détection d'une fuite (à la Stripe : `sk_live_...`, `sk_test_...`).

5.3 OAuth 2.0 et OpenID Connect

Définition

OAuth 2.0 (RFC 6749) est un protocole de *délégation d'autorisation* : il permet à une application tierce d'accéder à des ressources, au nom d'un utilisateur, *sans connaître son mot de passe*. Standard de fait du web depuis 2012.

OpenID Connect (OIDC) est une couche bâtie au-dessus d'OAuth 2.0 qui ajoute l'*authentification* (savoir qui est l'utilisateur via un ID Token).

Quatre rôles.

- **Resource Owner** : l'utilisateur final, propriétaire des données.
- **Client** : l'application tierce qui demande l'accès.
- **Authorization Server (AS)** : délivre les tokens (Auth0, Keycloak, Okta).
- **Resource Server (RS)** : votre API, qui détient les ressources protégées.

Quel *flow* (ou « grant type ») choisir ?

- **Authorization Code + PKCE** : pour app web, mobile, SPA avec utilisateur. Recommandé en 2026.
- **Client Credentials** : service à service sans utilisateur (M2M).
- **Device Authorization** : pour TV, console, IoT sans clavier (RFC 8628).
- **Refresh Token** : pour renouveler un access token expiré.

5.4 Le flow Authorization Code + PKCE (étape par étape)

Définition

PKCE (*Proof Key for Code Exchange*, RFC 7636) : extension d'OAuth 2.0 qui protège le client contre l'interception du code d'autorisation. Aujourd'hui obligatoire pour tous les clients publics (mobile, SPA).

1. Le client génère un secret aléatoire (*code_verifier*) et calcule son hash SHA-256 (*code_challenge*).
2. Il redirige l'utilisateur vers l'AS, en passant le `code_challenge`, le `client_id`, le `scope` et le `redirect_uri`.
3. L'utilisateur s'authentifie auprès de l'AS et donne son consentement.
4. L'AS redirige vers le `redirect_uri` avec un *authorization code* à usage unique.
5. Le client appelle l'AS pour échanger le code contre un `access_token` (et éventuellement un `refresh_token`), en présentant le `code_verifier` initial.
6. Le client appelle l'API protégée avec `Authorization: Bearer <access_token>`.

6 JWT : tokens auto-portés

Définition

JWT (*JSON Web Token*, RFC 7519) : format de jeton compact, encodé Base64URL, qui transporte des informations (*claims*) signées numériquement. Permet de transmettre une identité ou des autorisations sans appel à une base de données.

6.1 Structure d'un JWT

Un JWT est constitué de **trois segments** séparés par des points : `header.payload.signature`.

Exemple

```
HEADER : { "alg": "HS256", "typ": "JWT" }
PAYLOAD : { "sub": "42", "iss": "auth.acme.com",
            "aud": "api.acme.com", "exp": 1728479000,
            "scope": "read:articles" }
SIGNATURE : HMACSHA256(b64(header) + "." + b64(payload), SECRET)

Token complet (concaténé en Base64URL) :
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiJ9.7wXn-...
```

6.2 Claims standards

- `iss` (*issuer*) : qui a émis le token.
- `sub` (*subject*) : qui est concerné (souvent l'`user_id`).
- `aud` (*audience*) : à quel service le token est destiné.
- `exp` (*expiration*) : timestamp Unix d'expiration.
- `nbf` (*not before*) : timestamp à partir duquel le token est valide.
- `iat` (*issued at*) : timestamp d'émission.
- `jti` (*JWT ID*) : identifiant unique du token (utile pour la révocation).

6.3 Algorithmes

- **HS256/384/512** : HMAC symétrique. Secret partagé entre AS et RS.
- **RS256/384/512** : RSA asymétrique. AS signe avec clé privée, RS vérifie avec clé publique.
- **ES256/384/512** : ECDSA. Comme RSA mais clés plus courtes.
- **none** : pas de signature. *Doit être bloqué côté serveur.*

6.4 Pièges JWT — les classiques

À retenir

1. **`alg: none`** : toujours imposer une *whitelist* d'algos côté serveur.
2. **Pas d'expiration vérifiée** : toujours vérifier `exp`, `iss`, `aud`.
3. **Données sensibles dans le payload** : le JWT n'est pas chiffré, juste signé.
4. **Token long et sans rotation** : 5 à 15 minutes pour un access token.
5. **Stockage en localStorage** : vol par XSS. Préférer cookie HttpOnly.

7 Validation des entrées

7.1 Tout ce qui vient du client est suspect

La validation d'entrée est la *première défense*. Trois niveaux à appliquer en cascade :

1. **Syntaxique** : le format est-il correct ? (JSON parsable, types attendus, longueurs, regex).
2. **Sémantique** : les valeurs ont-elles du sens ? (énumérations, bornes métier, cohérence inter-champs).
3. **Métier** : l'opération est-elle autorisée ? (quotas, état valide, permissions).

À retenir

Règle d'or. Valider sur *liste blanche* (« seulement ces valeurs sont acceptées »), pas sur liste noire (« tout sauf ces valeurs »).

7.2 Valider avec un schéma déclaratif

Définition

JSON Schema (draft 2020-12) : méta-langage pour décrire la structure attendue d'un JSON. Permet de valider automatiquement chaque requête, et sert de contrat partagé entre back-end et front-end.

Exemple

Schéma de validation d'un article à créer :

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["title"],
  "properties": {
    "title": { "type": "string", "minLength": 3, "maxLength": 200 },
    "tags": { "type": "array",
              "items": { "enum": ["news","tech","fr"] } },
    "archived": { "type": "boolean", "default": false }
  }
}
```

Important. `additionalProperties: false` refuse tout champ inconnu : c'est la première barrière contre l'attaque *mass assignment* (un attaquant qui ajouterait `role: admin` dans le body).

8 Les injections

Définition

Injection : famille d'attaques où l'attaquant insère du code malveillant dans une donnée d'entrée, qui est ensuite interprété par un sous-système (BDD, parseur XML, shell, etc.).

8.1 Injection SQL

La plus connue. Cause : concaténation de chaînes pour construire des requêtes SQL.

Exemple

Vulnérable :

```
const sql = "SELECT * FROM users WHERE email = '"
          + req.body.email + "'";
// Payload : email = ' OR '1'='1' --
// -> renvoie tous les utilisateurs
```

Sûr (requête paramétrée) :

```
const sql = "SELECT * FROM users WHERE email = $1";
const rows = await db.query(sql, [req.body.email]);
// Le driver echappe la valeur, jamais interpretee comme du SQL
```

Règle absolue. Aucune concaténation de SQL avec des entrées utilisateur. Toujours requêtes préparées ou ORM avec bindings.

8.2 Autres injections à connaître

- **NoSQL injection** : opérateurs Mongo envoyés en JSON (`{ "$ne": null }`).
- **XML / XXE** : entités externes XML qui lisent des fichiers serveur ou émettent du SSRF.
- **Mass assignment** : un attaquant écrit un champ qu'il ne devrait pas (`role`, `is_admin`).
- **Command injection** : caractères spéciaux non échappés dans une commande shell.
- **LDAP / XPath / template injection** : mêmes principes, autres syntaxes.

9 Les outils du module

9.1 jwt.io

Outil

jwt.io (`https://jwt.io`) est un site web édité par Auth0 qui permet de :

- Décoder visuellement un JWT (header, payload, signature).
- Vérifier la signature avec un secret ou une clé publique.
- Tester la création de tokens dans plusieurs langages.
- Apprendre les algorithmes pris en charge par les bibliothèques.

Utilité pédagogique. Indispensable pour comprendre la structure d'un JWT et démontrer en live des attaques (par exemple `alg=none`).

9.2 Auth0

Outil

Auth0 (racheté par Okta en 2021) est une plateforme commerciale d'*Identity-as-a-Service* très utilisée. Permet de déléguer toute la gestion d'authentification (OAuth2, OIDC, MFA, social login, SSO) sans gérer soi-même un serveur d'identité.

Quand l'utiliser ? Quand on veut éviter d'écrire et de maintenir son propre IdP, et qu'on accepte la dépendance à un tiers (et son coût).

9.3 Keycloak

Outil

Keycloak (`https://www.keycloak.org`) est l'équivalent open source de Auth0, hébergé en interne. Édité par Red Hat. Implémente OAuth 2.0, OpenID Connect, SAML 2.0.

Quand l'utiliser ? Pour rester maître de ses données et de son infrastructure d'identité, ou pour des contraintes réglementaires (souveraineté). Demande une équipe ops compétente.

9.4 Ajv, Joi, Zod, Pydantic

Outil

Bibliothèques de validation par schéma :

- **Ajv** (Node.js) : implémentation très rapide de JSON Schema.
- **Joi** (Node.js) : DSL JavaScript pour décrire les schémas.
- **Zod** (TypeScript) : validation typée à la compilation et à l'exécution.
- **Pydantic** (Python) : standard de fait pour la validation en Python (utilisé par FastAPI).

Utilité. Toutes ces bibliothèques permettent de valider une donnée d'entrée contre un schéma, en une seule ligne. Indispensables pour empêcher mass assignment et injections paresseuses.

10 Synthèse

À retenir

- Authentification (≠) Autorisation. 401 = pas authentifié, 403 = pas le droit.
- OAuth 2.0 + PKCE = standard pour app web/mobile/SPA.
- JWT court (5–15 min) + refresh token, vérification stricte (alg, exp, iss, aud).
- Validation par schéma (JSON Schema), avec `additionalProperties: false`.
- Zéro concaténation SQL : toujours requêtes paramétrées ou ORM avec bindings.
- Liste blanche partout : énumérations, types, longueurs.

11 Lexique du module

ABAC Attribute-Based Access Control. Modèle d'autorisation basé sur des attributs.

API Key Chaîne identifiante longue et aléatoire pour authentifier un client programmatique.

Authentification Vérifier qui est l'utilisateur.

Authorization Server (AS) Composant OAuth qui émet les tokens.

Autorisation Décider ce que l'utilisateur peut faire.

BOLA Broken Object Level Authorization. Risque OWASP API1.

Bearer Type de token porté simplement par sa présentation (`Authorization: Bearer ...`).

CSRF Cross-Site Request Forgery. Forçage de requête depuis un site tiers.

Claim Affirmation contenue dans un token (sub, exp, role...).

Hash Empreinte unidirectionnelle d'une donnée. Utilisé pour stocker mots de passe et clés.

HMAC Hash-based Message Authentication Code. Signature symétrique.

Injection Famille d'attaques où une donnée d'entrée est interprétée comme du code.

JWE JSON Web Encryption. Variante chiffrée du JWT.

JWT JSON Web Token. Format de token signé.

Mass assignment Vulnérabilité où des champs non prévus sont écrits depuis le body.

OAuth 2.0 Protocole de délégation d'autorisation (RFC 6749).

OIDC OpenID Connect. Couche d'authentification au-dessus d'OAuth 2.0.

OWASP Open Web Application Security Project. Référence en sécurité applicative.

PKCE Proof Key for Code Exchange. Extension OAuth pour clients publics.

Problem Details Format standard d'erreur (RFC 7807/9457).

RBAC Role-Based Access Control. Modèle d'autorisation basé sur des rôles.

Refresh token Token long pour renouveler un access token court.

Resource Server (RS) Votre API protégée.

Scope Permission demandée par un client OAuth (`read:articles`).

SQLi Injection SQL.

Token Jeton porté pour authentification ou autorisation.

Validation par schéma Vérification automatique d'une donnée contre une description formelle.

XSS Cross-Site Scripting. Exécution de JS attaquant dans le contexte d'une victime.

12 Pour aller plus loin

- OWASP API Security Top 10 (2023).
- OWASP Cheat Sheets : REST Security, JWT, Authentication, Authorization.
- RFC 6749 (OAuth 2.0), RFC 7519 (JWT), RFC 7636 (PKCE), RFC 8252 (OAuth Native Apps).
- Justin Richer & Antonio Sanso, *OAuth 2 in Action*, Manning.
- `https://oauth.net` et `https://openid.net` pour les spécifications complètes.